

Five-Day Intern Project: AI PDF Chat Platform with MCP

Project Problem

Students and professionals often receive lengthy PDF documents but do not have enough time to read every page to find specific information. Build a small full-stack platform where users can create an account, verify their email, upload a PDF, and chat with its content using an LLM.

The system must use a React frontend, FastAPI backend, PostgreSQL with pgvector, and Docker. It must also include one simple MCP feature that does not require complicated third-party authentication.

Functional Requirements

1. User Authentication

Users must be able to:

- Sign up using their name, email, and password.
- Receive an OTP through Gmail SMTP.
- Verify their email using the OTP.
- Log in after email verification.
- View and update their profile.
- Change their password while logged in.
- Request a password-reset OTP through email.
- Reset their forgotten password using the OTP.
- Log out securely.

Use JWT authentication for protected APIs.

Passwords must be hashed and must never be stored as plain text. OTPs should expire after a limited time and should not be reusable.

Gmail SMTP credentials must be stored in environment variables. Students must use a Gmail App Password, not their actual Gmail password.

2. PDF Management

After logging in, users must be able to:

- Upload one or more PDF files.
- View their uploaded PDF files.
- Select a PDF for conversation.
- Delete their own PDF files.
- View previous questions and answers for a selected PDF.

For this five-day project, supporting text-based PDFs is sufficient. OCR for scanned PDFs is not required.

3. PDF Processing and Vector Storage

When a user uploads a PDF, the backend must:

1. Validate that the uploaded file is a PDF.
2. Extract text from the PDF.
3. Divide the text into manageable chunks.
4. Generate an embedding for each chunk.
5. Store the chunks and embeddings in PostgreSQL using pgvector.
6. Connect every stored chunk to the correct user and PDF.

Users must never be able to access or chat with PDFs uploaded by another user.

4. Chat with PDF

The platform must provide a chatbot where the user can ask questions about the selected PDF.

For every question, the system should:

1. Generate an embedding for the user's question.
2. Search pgvector for the most relevant chunks from the selected PDF.
3. Send the relevant content and question to the LLM.
4. Return a clear answer based only on the PDF content.
5. Save the question and answer in the database.

If the answer is not available in the PDF, the chatbot should clearly say:

“I could not find this information in the selected document.”

The chatbot should not invent information that is not supported by the uploaded PDF.

5. Simple MCP Feature

Create a small local MCP server with one tool:

```
get_word_definition(word)
```

The chatbot can use this tool when the user asks for the meaning of a technical or unfamiliar word found in the PDF.

Example:

“What does authentication mean?”

The LLM should call the MCP tool and return the definition.

The MCP server may use a free public dictionary API or a small local dictionary. It does not require OAuth or a separate user-login system. No additional MCP authentication is required because the MCP server will run locally inside the Docker environment.

Students must demonstrate at least one successful MCP tool call.

6. React Frontend

Create a simple and responsive React interface with the following pages:

- Sign-up page
- Email OTP verification page
- Login page
- Forgot-password page
- Reset-password page
- User profile page
- PDF dashboard
- PDF upload section
- PDF chatbot page
- Chat-history section

Advanced design is not required. The focus should be on functionality, clear navigation, validation messages, loading indicators, and error handling.

7. FastAPI Backend

The FastAPI backend should contain separate modules for:

- Authentication
- User profiles
- Email and OTP handling
- PDF upload and processing
- Embedding generation
- Vector search
- Chat history
- LLM integration
- MCP integration

Use Pydantic models for request and response validation.

8. Database

Use PostgreSQL with the pgvector extension.

The database should contain tables for:

- Users
- Email OTPs
- Password-reset OTPs
- PDF documents
- PDF chunks and embeddings
- Chat messages

Appropriate relationships should be created between users, documents, chunks, and chat messages.

9. Docker

The complete application must run through Docker Compose.

Create containers for:

- React frontend
- FastAPI backend
- PostgreSQL with pgvector
- MCP server

The complete system should start with:

`docker compose up --build`

Five-Day Development Plan

Day 1: Authentication and Database

- Set up FastAPI, React, and PostgreSQL.
- Create the user model.
- Implement sign-up and password hashing.
- Send OTPs through Gmail SMTP.
- Implement email verification and login.

Day 2: User and Password Management

- Implement JWT authentication.
- Add profile viewing and updating.
- Add change-password functionality.
- Implement forgot-password and reset-password OTP flows.

Day 3: PDF Upload and Vector Processing

- Implement PDF upload and validation.
- Extract and divide PDF text.
- Generate embeddings.
- Store document chunks in pgvector.
- Display uploaded documents on the frontend.

Day 4: PDF Chatbot and MCP

- Implement vector similarity search.

- Connect the retrieved content to the LLM.
- Build the PDF chatbot interface.
- Store chat history.
- Create and connect the word-definition MCP tool.

Day 5: Docker, Testing and Documentation

- Dockerize all services.
- Test the complete application.
- Fix errors and validation issues.
- Write the README.
- Record the demonstration video.

Project Limitations

To keep the project achievable within five days:

- OCR is not required.
- Social login is not required.
- Payment integration is not required.
- Advanced user roles are not required.
- Live chat streaming is optional.
- Only one LLM provider is required.
- Only one MCP tool is required.
- A simple React design is acceptable.
- Deployment to a public cloud is optional.

Required Deliverables

Students must submit:

- Complete source code
- GitHub repository link
- React frontend

- FastAPI backend
- PostgreSQL and pgvector configuration
- MCP server
- Dockerfiles
- Docker Compose file
- .env.example without real credentials
- Database schema or ER diagram
- README with setup instructions
- Three-to-five-minute demonstration video

Demonstration Requirements

During the demonstration, the student must:

1. Register a new user.
2. Receive and verify an email OTP.
3. Log in successfully.
4. Update the user profile.
5. Demonstrate the forgot-password flow.
6. Upload a PDF.
7. Ask at least two questions about the PDF.
8. Ask one question whose answer is unavailable in the PDF.
9. Demonstrate the MCP word-definition tool.
10. Start the complete application through Docker Compose.

Students must understand and explain their implementation. Real passwords, Gmail App Passwords, API keys, and other credentials must not be uploaded to GitHub.